EXPERIMENT NO. 8

CUSTOMER CHURN / EMPLOYEE ATTRITION PREDICTION

USING DECISION TREE CLASSIFICATION

Name : Dev Sharma

UID : CU24250269

Course : B.Tech CSE

Section : CSE "B"

Roll No. : 17

AIM

To build and evaluate a Decision Tree Classification model for predicting employee attrition and analyze the factors influencing employee retention.

STEP 1: IMPORT REQUIRED LIBRARIES

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from tkinter import Tk
from tkinter.filedialog import askopenfilename

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    confusion_matrix,
    ConfusionMatrixDisplay,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)
```

STEP 2: SELECT AND LOAD DATASET

```
from google.colab import files
import io
uploaded = files.upload()
if not uploaded:
print("No file selected.")
else:
for filename in uploaded.keys():
# Assuming the uploaded file is a CSV
df = pd.read_csv(io.StringIO(uploaded[filename].decode('utf-8')))
print("Dataset loaded successfully!")
print("Shape:", df.shape)
```

STEP 3: DATA PREPROCESSING

```
df = df.loc[:, ~df.columns.str.contains("^Unnamed")]

# Remove extra spaces from column names
df.columns = df.columns.str.strip()

# Remove duplicate records
df = df.drop_duplicates()

# Convert categorical columns into numerical values
# Attrition: Yes = 1, No = 0
df["Attrition"] = df["Attrition"].map({"Yes": 1, "No": 0})

# Convert categorical feature columns using label encoding
categorical_columns = df.select_dtypes(include=["object"]).columns

for column in categorical_columns:
    df[column] = df[column].astype("category").cat.codes

# Handle missing values
for column in df.columns:
```

```
    if df[column].isnull().any():
        df[column] = df[column].fillna(df[column].median())
```

STEP 4: SELECT FEATURES AND TARGET

```
y = df["Attrition"]

# Remove target and columns that do not contribute useful information
columns_to_remove = [
    "Attrition",
    "EmployeeNumber",
    "EmployeeCount",
    "Over18",
    "StandardHours"
]

X = df.drop(columns=columns_to_remove, errors="ignore")

# Keep numerical features
X = X.select_dtypes(include=np.number)
```

STEP 5: SPLIT DATA INTO TRAINING AND TESTING SETS

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

STEP 6: CREATE AND TRAIN DECISION TREE MODEL

```
model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=5,
    random_state=42
)

model.fit(X_train, y_train)
```

STEP 7: PREDICT ATTRITION

```
y_pred = model.predict(X_test)
```

STEP 8: CONFUSION MATRIX

```
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["No Attrition", "Attrition"]
).plot()

plt.title("Confusion Matrix - Decision Tree")
plt.show()
```

STEP 9: MODEL PERFORMANCE

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, zero_division=0)
recall = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)

print("\nModel Performance:")
print("Accuracy :", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall   :", round(recall, 4))
print("F1-Score :", round(f1, 4))
```

STEP 10: FEATURE IMPORTANCE

```
feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
})

feature_importance = feature_importance.sort_values(
    by="Importance",
    ascending=False
)

print("\nTop 10 Important Features:")
print(feature_importance.head(10))

# Plot top 10 important features
top_features = feature_importance.head(10).sort_values(
    by="Importance"
)

plt.figure(figsize=(10, 6))

plt.barh(
    top_features["Feature"],
    top_features["Importance"]
)

plt.xlabel("Importance")
plt.ylabel("Features")
plt.title("Top 10 Features Influencing Employee Attrition")
plt.tight_layout()
plt.show()
```

STEP 11: VISUALIZE DECISION TREE

```
plt.figure(figsize=(24, 12))

plot_tree(
    model,
    feature_names=X.columns,
    class_names=["No Attrition", "Attrition"],
    filled=True,
    rounded=True,
    fontsize=8
)

plt.title("Decision Tree for Employee Attrition Prediction")
plt.show()
```

STEP 12: ATTRITION ANALYSIS

```
attrition_rate = df["Attrition"].mean() * 100

print("\nEmployee Attrition Rate:", round(attrition_rate, 2), "%")
```

QUESTIONS AND ANSWERS

==========

Q1. What is the difference between classification and regression?

Classification predicts discrete categories or classes. Example: Attrition = Yes or No.

Regression predicts continuous numerical values. Example: Predicting employee salary.

Q2. How does a Decision Tree work?

A Decision Tree repeatedly divides the dataset using feature conditions. Each division creates branches and eventually reaches leaf nodes containing the predicted class.

Q3. What is the difference between Gini Index and Entropy?

Both Gini Index and Entropy measure impurity in a dataset.

Gini Index: Measures how often a randomly selected sample would be incorrectly classified.

Entropy: Measures the uncertainty or disorder in the dataset.

Lower impurity indicates a better split.

Q4. What are the four components of a Confusion Matrix?

True Positive (TP) True Negative (TN) False Positive (FP) False Negative (FN)

Q5. What are Accuracy, Precision, Recall and F1-Score?

Accuracy = (TP + TN) / (TP + TN + FP + FN)

Precision = TP / (TP + FP)

Recall = TP / (TP + FN)

F1-Score = 2 * (Precision * Recall) / (Precision + Recall)

Accuracy measures overall correctness. Precision measures correctness of positive predictions. Recall measures how many actual positive cases were identified. F1-Score balances Precision and Recall.

Q6. What is overfitting and how can it be reduced?

Overfitting occurs when a Decision Tree learns the training data too closely and performs poorly on unseen data.

It can be reduced by:

Limiting max_depth Increasing min_samples_split Increasing min_samples_leaf Pruning the tree Using cross-validation

Q7. Why is employee attrition prediction important?

Predicting employee attrition helps organizations identify employees who may leave. HR departments can take preventive actions, reduce employee turnover and improve workforce planning.

Q8. What are the advantages and limitations of Decision Trees?

Advantages:

Easy to understand Easy to visualize Requires little preprocessing Can handle numerical and categorical features

Limitations:

Can overfit Small data changes can change the tree Deep trees may become complex

Q9. Give three real-world applications of classification other than employee churn prediction.

= Spam email detection

Disease diagnosis

Credit card fraud detection

Q10. How can the insights from this model improve retention and profitability?

HR departments can identify the important factors associated with employee attrition and take targeted actions such as improving working conditions, reviewing compensation, providing training, improving job satisfaction and creating employee engagement programs.

Reducing unnecessary employee turnover can lower recruitment and training costs and improve organizational productivity.

==========

END OF EXPERIMENT